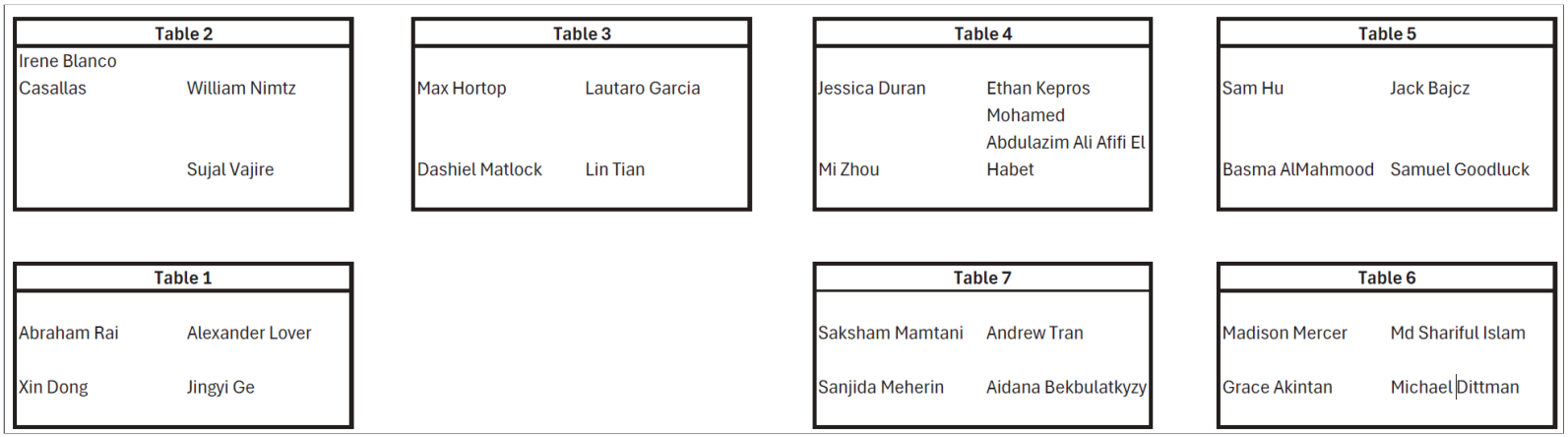


Table 2	
Irene Blanco Casallas	William Nimtz
	Sujal Vajire

Table 3	
Max Hortop	Lautaro Garcia
Dashiel Matlock	Lin Tian

Table 4	
Jessica Duran	Ethan Kepros
Mi Zhou	Mohamed Abdulazim Ali Afifi El Habet

Table 5	
Sam Hu	Jack Bajcz
Basma AlMahmood	Samuel Goodluck

Table 1	
Abraham Rai	Alexander Lover
Xin Dong	Jingyi Ge

Table 7	
Saksham Mamtani	Andrew Tran
Sanjida Meherin	Aidana Bekbulatkyzy

Table 6	
Madison Mercer	Md Shariful Islam
Grace Akintan	Michael Dittman

CMSE 801 - Day 13

Using Pandas to make an informed assessment based on data

Exploring the Flint water crisis data

Feb 24, 2026

General Course Announcements

- Homework 2 is due **this Saturday**, Feb 28, by 11:59pm.
- Quiz 2 will take place on **this Thursday** (Feb 26).
    - The content of the quiz will cover concepts and assess knowledge that you demonstrated in Homework 2.
    - AI assistant is online.
- Homework 3 will be posted today and is due **March 21**.

Questions/comments from Day 13 PCA

- Most common challengs were:
    - plotting: subplot, googling syntax, looping etc
    - using log scale
    - masking, selecting data

Before we march on, let's import the modules we might use.

In [1]:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

Let's review some of the plotting components and log scale

In [3]:

```
# First we need to load some data to work with
gdp = pd.read_csv("GDP_Cleaned.csv")
gdp.head(10)
```

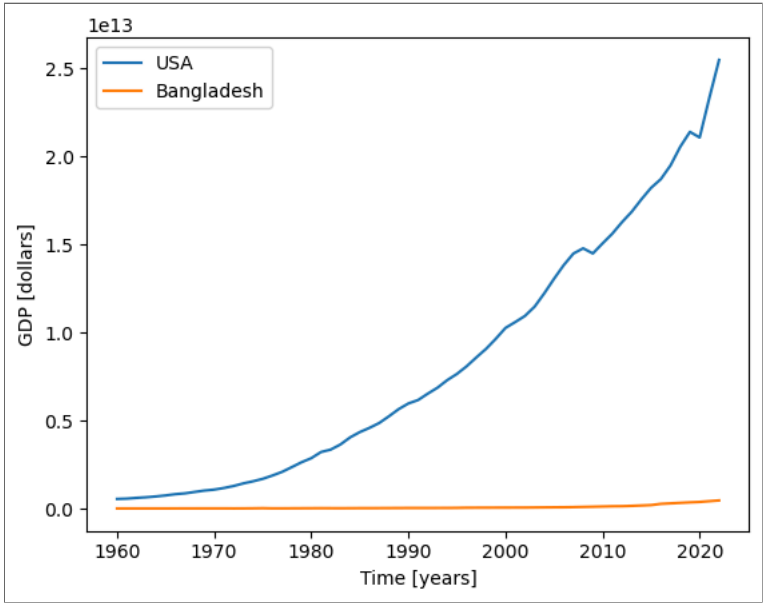
Out[3]:

	Year	Africa Eastern and Southern	Africa Western and Central	Australia	Austria	Burundi	Belgium	
0	1960	2.112502e+10	1.044764e+10	1.860656e+10	6.592694e+09	1.960000e+08	1.165872e+10	1
1	1961	2.161623e+10	1.117321e+10	1.968288e+10	7.311750e+09	2.030000e+08	1.240015e+10	2
2	1962	2.350628e+10	1.199053e+10	1.992256e+10	7.756110e+09	2.135000e+08	1.326402e+10	2
3	1963	2.804836e+10	1.272769e+10	2.153984e+10	8.374175e+09	2.327500e+08	1.426002e+10	1
4	1964	2.592067e+10	1.389811e+10	2.380112e+10	9.169984e+09	2.607500e+08	1.596011e+10	2
5	1965	2.947210e+10	1.492979e+10	2.597616e+10	9.994071e+09	1.589950e+08	1.737146e+10	2
6	1966	3.201437e+10	1.591084e+10	2.730784e+10	1.088768e+10	1.654446e+08	1.865188e+10	3
7	1967	3.326951e+10	1.451058e+10	3.044272e+10	1.157943e+10	1.782971e+08	1.999204e+10	3
8	1968	3.632779e+10	1.496824e+10	3.271409e+10	1.244063e+10	1.832000e+08	2.137635e+10	3
9	1969	4.163897e+10	1.697932e+10	3.668337e+10	1.358280e+10	1.902057e+08	2.371074e+10	3

10 rows × 123 columns

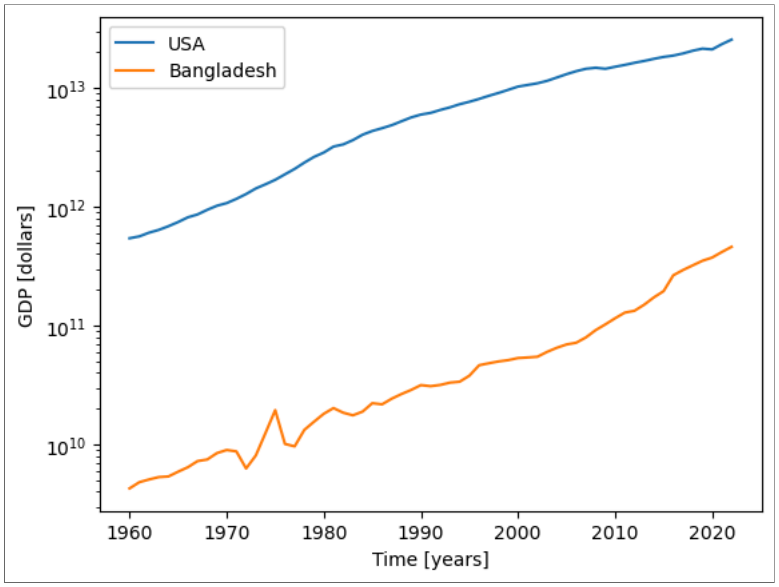
In [4]:

```
plt.plot(gdp["Year"],gdp['United States'],label="USA")
plt.plot(gdp["Year"],gdp['Bangladesh'], label="Bangladesh")
plt.xlabel("Time [years]")
plt.ylabel("GDP [dollars]")
plt.legend();
```



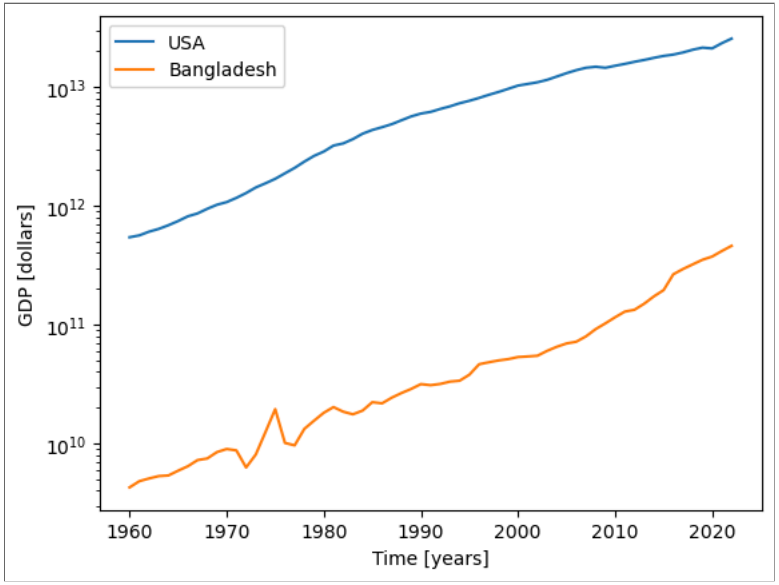
In [6]:

```
plt.plot(gdp["Year"],gdp['United States'],label="USA")
plt.plot(gdp["Year"],gdp['Bangladesh'], label="Bangladesh")
plt.yscale("log")
plt.xlabel("Time [years]")
plt.ylabel("GDP [dollars]")
plt.legend();
```



In [7]:

```
plt.semilogy(gdp["Year"],gdp['United States'],label="USA")
plt.semilogy(gdp["Year"],gdp['Bangladesh'], label="Bangladesh")
plt.xlabel("Time [years]")
plt.ylabel("GDP [dollars]")
plt.legend();
```



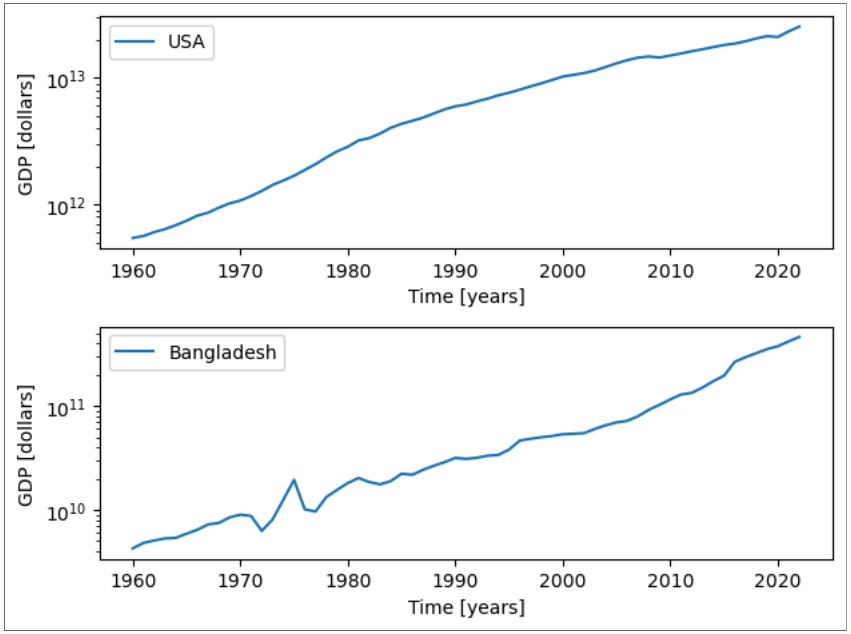
Using subplots

In [9]:

```
plt.figure(1)
plt.subplot(211)
plt.semilogy(gdp["Year"],gdp['United States'],label="USA")
plt.xlabel("Time [years]")
plt.ylabel("GDP [dollars]")
plt.legend();

plt.subplot(212)
plt.semilogy(gdp["Year"],gdp['Bangladesh'], label="Bangladesh")
plt.xlabel("Time [years]")
plt.ylabel("GDP [dollars]")
plt.legend();

plt.tight_layout();
```


USA
GDP [dollars]
10^{13}
10^{12}
1960
1970
1980
1990
2000
2010
2020
Time [years]
Bangladesh
GDP [dollars]
10^{11}
10^{10}
1960
1970
1980
1990
2000
2010
2020
Time [years]

Let's do a little bit of experimentation with masks

In [10]:

```
# Let's generate some fake data
my_data = np.random.randint(low=0,high=100,size=100)
print(my_data)
```

```
[18 90 87 21 79 26 84 82 12  2 57 96 87 99 76 23 28 91 73 99 99 20 90 99
 58 63 63 18 46 55 24 94 90 56 62 28 66 51 93 57 92 39 47 28 67 42 91 70
 13 50 12 23 15 51 98 18 61  1  4 58 21 37 47 62 48 58 52 19  5 64 16 43
 88 95 91 11 23 60 94 89 20 16 61 41 90 45 55 37 56 32 55 43  5 61 68 36
 23 91 31 66]
```

In [12]:

```
mask = my_data > 25
# print(mask)
print(my_data[mask])
print("Number of values greater than 25:", len(my_data[mask]))
```

```
[90 87 79 26 84 82 57 96 87 99 76 28 91 73 99 99 90 99 58 63 63 46 55 94
 90 56 62 28 66 51 93 57 92 39 47 28 67 42 91 70 50 51 98 61 58 37 47 62
 48 58 52 64 43 88 95 91 60 94 89 61 41 90 45 55 37 56 32 55 43 61 68 36
 91 31 66]
Number of values greater than 25: 75
```

In [13]:

```
mask = my_data == 25
print(mask)
print(my_data[mask])
print("Number of values equal to 25:", len(my_data[mask]))
```

```
[False False False False False False False False False False False False
 False False False False False False False False False False False False
 False False False False False False False False False False False False
 False False False False False False False False False False False False
 False False False False False False False False False False False False
 False False False False False False False False False False False False
 False False False False False False False False False False False False
 False False False False False False False False False False False False
 False False False False]
[]
Number of values equal to 25: 0
```

In [14]:

```
mask_both = (my_data > 25) & (my_data < 75) # this uses a bitwise "AND"
masked_data = my_data[mask_both]
print(masked_data)
print("Number of values between 25 and 75:", len(masked_data))
```

```
[26 57 28 73 58 63 63 46 55 56 62 28 66 51 57 39 47 28 67 42 70 50 51 61
 58 37 47 62 48 58 52 64 43 60 61 41 45 55 37 56 32 55 43 61 68 36 31 66]
Number of values between 25 and 75: 48
```

This is using something we haven't seen/talked about, which are **"bitwise" operators** in Python.

We can using masks with Pandas data as well!

In [15]:

```
from io import StringIO
# read in some data on heart disease
heart_disease_data = pd.read_json(StringIO("""[{"observation_number":0,"age":67,"sex":1,"cp":4,"trestbps":160,"chol":28
heart_disease_data.head()
```

Out[15]:

	observation_number	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	num
0	0	67	1	4	160	286	0	2	108	1	1.5	2	3.0	3.0	2
1	1	67	1	4	120	229	0	2	129	1	2.6	2	2.0	7.0	1
2	2	37	1	3	130	250	0	0	187	0	3.5	3	0.0	3.0	0
3	3	41	0	2	130	204	0	2	172	0	1.4	1	0.0	3.0	0
4	4	56	1	2	120	236	0	0	178	0	0.8	1	0.0	3.0	0

In [16]:

```
over_60_mask = heart_disease_data['age'] > 60
over_60_data = heart_disease_data[over_60_mask]
over_60_data.head(10)
```

Out[16]:

	observation_number	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	num
0	0	67	1	4	160	286	0	2	108	1	1.5	2	3.0	3.0	2
1	1	67	1	4	120	229	0	2	129	1	2.6	2	2.0	7.0	1
5	5	62	0	4	140	268	0	2	160	0	3.6	3	2.0	3.0	3
7	7	63	1	4	130	254	0	2	147	0	1.4	2	1.0	7.0	2
19	19	64	1	1	110	211	0	2	144	1	1.8	2	0.0	3.0	0
26	26	66	0	1	150	226	0	0	114	0	2.6	3	0.0	3.0	0
29	29	69	0	1	140	239	0	0	151	0	1.8	1	2.0	3.0	0
31	31	64	1	3	140	335	0	0	158	0	0.0	1	0.0	3.0	1
38	38	61	1	3	150	243	1	0	137	1	1.0	2	0.0	3.0	0
39	39	65	0	4	150	225	0	2	114	0	1.0	2	3.0	7.0	4

In [17]:

```
# Again, I can be lazier about this
over_60_data = heart_disease_data[heart_disease_data['age'] > 60]
over_60_data.head(10)
```

Out[17]:

	observation_number	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	num
0	0	67	1	4	160	286	0	2	108	1	1.5	2	3.0	3.0	2
1	1	67	1	4	120	229	0	2	129	1	2.6	2	2.0	7.0	1
5	5	62	0	4	140	268	0	2	160	0	3.6	3	2.0	3.0	3
7	7	63	1	4	130	254	0	2	147	0	1.4	2	1.0	7.0	2
19	19	64	1	1	110	211	0	2	144	1	1.8	2	0.0	3.0	0
26	26	66	0	1	150	226	0	0	114	0	2.6	3	0.0	3.0	0
29	29	69	0	1	140	239	0	0	151	0	1.8	1	2.0	3.0	0
31	31	64	1	3	140	335	0	0	158	0	0.0	1	0.0	3.0	1
38	38	61	1	3	150	243	1	0	137	1	1.0	2	0.0	3.0	0
39	39	65	0	4	150	225	0	2	114	0	1.0	2	3.0	7.0	4

Goals for today

- Load and explore data from the Flint water crisis using **Pandas**.
- By analyzing and visualizing the data in a variety of ways, make an assessment as to whether the lead levels in the Flint water supplied violated the EPA guidelines.

Looking forward - Day 14 PCA: Fitting models, making predictions, evaluating fits

- You'll spend some more time thinking about what it means to "fit a model" to data and try to devise your own algorithm for calculating a "goodness of fit"
- You'll also explore the `curve_fit` tool from SciPy, which can be used to fit models to data.